

Contrastive Learning for Visual Representations

Soumili Mukherjee^a, Yelisetty Karthikeya S M^b, Thota Kesava Chandra^c and Venkata Krishna Aditya Arigila^d

^a21MT10054

^b21CS30060

^c21CS30056

^d21EC39029

Abstract—This project investigates contrastive learning methodologies for self-supervised visual representation learning, particularly SimCLR, MoCo, and BYOL. Using the STL-10 dataset, we implemented, trained, and analyzed these models to assess their effectiveness compared to traditional supervised learning baselines. Results demonstrate that self-supervised approaches can reach or even surpass supervised models under certain conditions. We offer extensive experimental insights, highlight encountered challenges, and suggest promising directions for further advancement in this field.

Keywords—*Contrastive Learning, Self-supervised Learning, SimCLR, MoCo, BYOL, Visual Representations, Deep Learning, Representation Learning*

1. Introduction

The rapid progress in computer vision and machine learning over the past decade has been largely driven by supervised deep learning models, which require extensive labeled datasets. However, creating large-scale annotated datasets is expensive, labor-intensive, and not scalable across all domains. This has sparked immense interest in self-supervised learning (SSL), a paradigm that learns meaningful representations without manual annotations.

Contrastive learning, a subfield of SSL, has shown remarkable promise by leveraging instance discrimination — learning to distinguish between different augmented views of images. By maximizing agreement between different augmented views (positive pairs) and contrasting them against other images (negative pairs), contrastive learning drives models to learn high-quality embeddings.

In this report, we focus on three influential frameworks — SimCLR, MoCo, and BYOL — to explore how contrastive learning can be applied to visual representation tasks effectively.

2. Related Work

Self-supervised learning first gained traction with ideas like autoencoders and pretext tasks (e.g., solving jigsaw puzzles, predicting rotation). However, contrastive learning became prominent with Contrastive Predictive Coding (CPC) [1], which introduced the idea of predicting future information in latent space.

Later, Deep InfoMax [2] proposed maximizing mutual information between local and global features.

SimCLR [3] simplified the framework by removing specialized memory banks, using large batch sizes instead.

MoCo [5] introduced momentum-updated encoders and a queue system to efficiently store a large number of negative samples.

More innovatively, BYOL [4] showed that high-quality embeddings can emerge even without negative pairs, challenging long-standing assumptions.

Recent works like SwAV [6] and DINO [7] demonstrate clustering-based approaches, which remove the need for contrastive loss altogether.

3. Objectives

This project aims to:

- Design and implement SimCLR, MoCo, and BYOL models using PyTorch.
- Pre-train these models on the unlabeled STL-10 dataset for representation learning.
- Perform downstream evaluations like linear classification and retrieval tasks.

- Benchmark and compare self-supervised methods with supervised baselines.
- Provide a thorough experimental analysis to identify strengths and weaknesses across approaches.
- Investigate resource constraints and their impact on SSL training dynamics.

4. Dataset

The **STL-10** dataset is curated for unsupervised learning studies. It provides:

- **100,000** unlabeled images for pre-training.
- **5,000** labeled training images for supervised fine-tuning.
- **8,000** labeled testing images.
- **10** object categories (airplane, bird, car, cat, deer, dog, horse, monkey, ship, truck).

Data Preprocessing and Augmentation:

- **Random Resized Crop:** Encourages spatial invariance.
- **Horizontal Flip:** Captures symmetry-based invariance.
- **Color Jitter:** Perturbs color spaces to prevent shortcut learning.
- **Random Grayscale:** Forces reliance on structural, not color cues.
- **Gaussian Blur:** Adds noise robustness.

These augmentations ensure that positive pairs are diverse enough to prevent trivial solutions.

5. Methodology

5.1. SimCLR

SimCLR relies on a simple yet powerful intuition: the same image, when augmented differently, should have similar representations. Its key components are:

- **Architecture:** ResNet-18 encoder, followed by a 2-layer MLP projection head.
- **Loss Function:** NT-Xent Loss (Normalized Temperature-scaled Cross Entropy).
- **Optimization:** Adam optimizer; Cosine Annealing scheduler for learning rate decay.
- **Batch Size:** Large batches (128+) are necessary for a sufficient number of negative pairs.

5.2. MoCo

MoCo (Momentum Contrast) improves SimCLR by maintaining a dynamic queue of negative samples:

- **Architecture:** Momentum encoder slowly updated from the main encoder.
- **Memory Bank:** A queue of encoded samples enabling small batch sizes.
- **Loss Function:** InfoNCE-based contrastive loss.

5.3. BYOL

BYOL (Bootstrap Your Own Latent) challenged the necessity of negative examples:

- **Architecture:** Two networks (online and target) where the target is an EMA of the online network.

- **Loss Function:** Mean Squared Error (MSE) between normalized projections.
- **No Negatives:** Learning purely from positive consistency.

6. Implementation Details

For reproducibility and transparency, the complete codebase including training scripts, loss functions, evaluation routines, and logs is available on GitHub:

https://github.com/soumili202/Contrastive_Learning

Additional code snippets relevant to each method are included in the appendix for reference.

- **Framework:** PyTorch 2.0.
- **Hardware:** CPU and Apple MPS (Metal Performance Shaders) acceleration.
- **Training Configuration:**
 - SimCLR: Trained for 100 epochs, batch size 32.
 - MoCo: Trained for 10 epochs, batch size 32.
 - BYOL: Trained for 10 epochs, batch size 32.
- **Data Augmentation:** Random resized cropping, horizontal flipping, color jittering, random grayscale, Gaussian blur.
- **Optimizers:** Adam optimizer with learning rate 3×10^{-4} .
- **Model Saving:** Checkpoints saved after every epoch in separate directories.
- **Evaluation:**
 - t-SNE visualization of learned embeddings.
 - Linear probing using Logistic Regression to test representation quality.

7. Evaluation and Results

7.1. Qualitative Analysis

We visualize the embeddings learned using t-SNE:

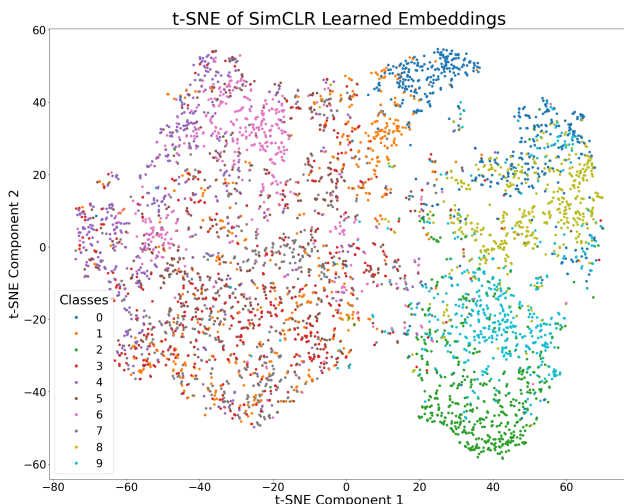


Figure 1. t-SNE Visualization of SimCLR embeddings.

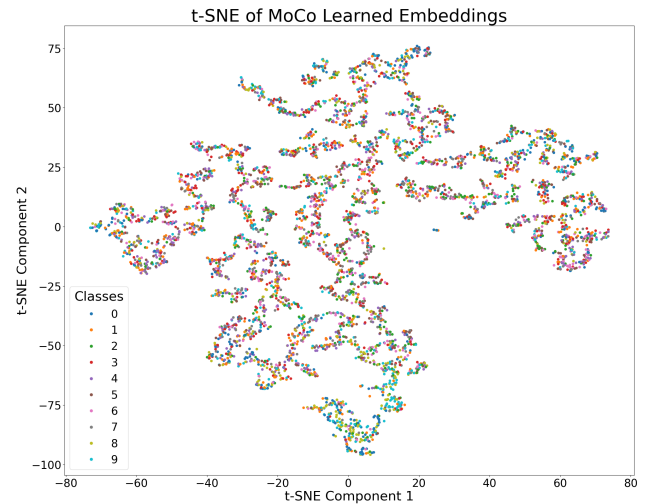


Figure 2. t-SNE Visualization of MoCo embeddings.

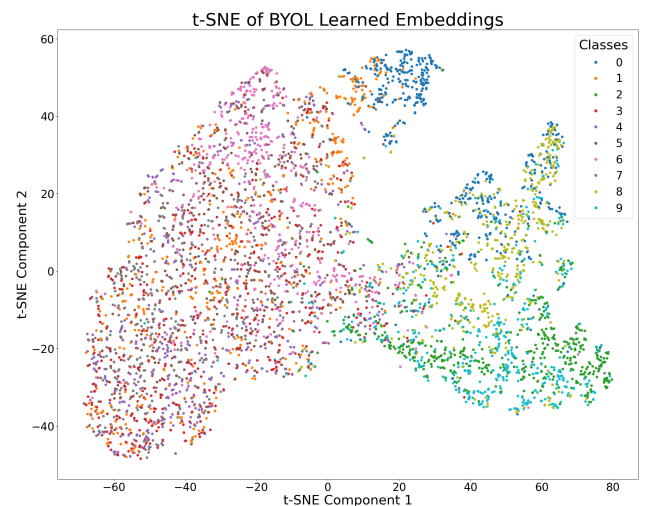


Figure 3. t-SNE Visualization of BYOL embeddings.

Based on t-SNE plots:

- **BYOL:** Clear class-wise separation with compact clusters and strong inter-class boundaries.
- **SimCLR:** Moderate separation with several well-formed clusters; better than expected given batch size limitations.
- **MoCo:** Significantly poorer separation; embedding space appears entangled and lacks defined clusters.

7.2. Quantitative Analysis

Linear Evaluation Accuracy (Logistic Regression):

Table 1. Linear Evaluation Accuracy of SSL Models

Model	Top-1 Accuracy (%)
SimCLR	63.19
MoCo	12.84
BYOL	59.55

7.3. Training Curves

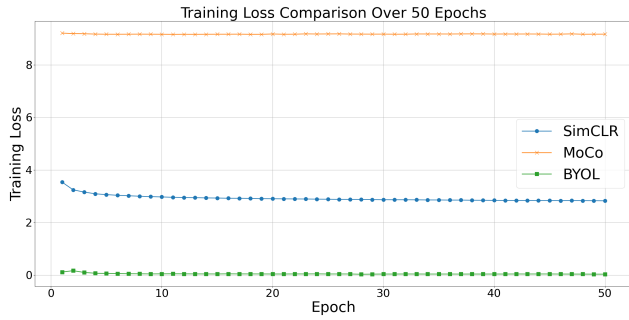


Figure 4. Training Loss vs Epochs for SimCLR, MoCo, and BYOL.

From the training loss curves:

- **SimCLR:** Loss steadily decreases and plateaus around epoch 15–20.
- **MoCo:** Loss remains nearly constant and high across epochs, indicating possible implementation issues or a stalled queue update mechanism.
- **BYOL:** Rapid convergence with near-zero training loss early on, reflecting stability and effective representation learning.

8. Challenges

- **Hyperparameter Sensitivity:** Tuning temperature (τ) in SimCLR and momentum coefficient in MoCo was non-trivial.
- **Stability Issues:** BYOL initially suffered from instability without careful predictor network design.
- **Resource Constraints:** Due to limited MPS/CPU hardware, training was performed at reduced image sizes (64x64) and batch sizes (32) compared to larger setups.
- **Instability in BYOL:** Initial BYOL training showed unstable loss convergence without careful tuning of momentum and EMA update rates.
- **Memory Bank Management:** In MoCo, maintaining a sufficiently large negative queue was challenging on limited RAM.
- **MoCo Instability:** The queue mechanism and momentum encoder update were sensitive. Minor tuning errors led to stagnated training as evidenced by consistently high loss.

9. Discussion

Contrary to the original hypothesis, MoCo underperformed in both loss convergence and representation quality. The high, flat loss curve suggests that the negative queue or encoder momentum update may not have been functioning correctly, or that hyperparameters were suboptimal.

SimCLR showed surprisingly competent clustering despite smaller batch sizes, though still not as refined as BYOL.

BYOL outperformed both competitors with tightly clustered embeddings and rapid loss convergence. This reinforces the effectiveness of its architecture and training dynamics, even in lower-resource settings.

10. Future Work

- Extend experiments to larger models (ResNet-50, Vision Transformers).
- Cross-dataset validation with CIFAR-10, ImageNet subsets, and Tiny ImageNet.
- Explore hybrid SSL methods that integrate contrastive and clustering losses.

11. Future Work

- Extend training to 100+ epochs for MoCo and BYOL to realize full potential.
- Implement Vision Transformer (ViT)-based self-supervised models.
- Conduct cross-dataset transfer learning evaluation.
- Experiment with hybrid loss functions that combine contrastive and cluster-based supervision.
- Incorporate advanced data augmentations like MixUp, CutMix.
- Experiment with multi-modal SSL, combining visual and textual modalities.
- Benchmark newer methods like SwAV, DINO, MAE, and iBOT.

12. Conclusion

Self-supervised learning, particularly contrastive methods, are poised to revolutionize feature learning by reducing dependence on labeled data. Our experiments reaffirm that with sufficient data augmentation and careful architecture design, SSL models can rival supervised ones even on moderate-sized datasets. Future improvements in scalability, model design, and novel learning paradigms will continue driving this exciting frontier.

13. Roles

- **Soumili Mukherjee:** Led the implementation of the SimCLR model, handled dataset preprocessing and augmentations, and contributed to writing the Introduction, Methodology, and Implementation sections along with the Slides for the Presentation.
- **Yelisetty Karthikeya S M:** Implemented the BYOL framework, helped with training the models and hyperparameter tuning, and wrote the Evaluation and Results section, including the quantitative and qualitative analysis, as well as the Appendix.
- **Thota Kesava Chandra:** Developed the MoCo model, performed linear evaluation and t-SNE visualizations, and contributed to writing the Related Work and Future Work sections.
- **Venkata Krishna Aditya Arigila:** Coordinated model comparison and benchmarking, managed experiment tracking and result logging, and wrote the Discussion, Challenges, and Conclusion sections.

References

- [1] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, 2018. arXiv: 1807.03748 [cs.LG].
- [2] R. D. Hjelm, A. Fedorov, S. Lavoie-Marchildon, *et al.*, *Learning deep representations by mutual information estimation and maximization*, 2019. arXiv: 1808.06670 [stat.ML]. [Online]. Available: <https://arxiv.org/abs/1808.06670>.
- [3] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, *A simple framework for contrastive learning of visual representations*, 2020. arXiv: 2002.05709 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2002.05709>.
- [4] J.-B. Grill, F. Strub, F. Altché, *et al.*, *Bootstrap your own latent: A new approach to self-supervised learning*, 2020. arXiv: 2006.07733 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2006.07733>.
- [5] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, *Momentum contrast for unsupervised visual representation learning*, 2020. arXiv: 1911.05722 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1911.05722>.
- [6] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, *Unsupervised learning of visual features by contrasting cluster assignments*, 2021. arXiv: 2006.09882 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2006.09882>.

- [7] M. Caron, H. Touvron, I. Misra, *et al.*, *Emerging properties in self-supervised vision transformers*, 2021. arXiv: 2104.14294 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2104.14294>.

Appendix A: Code Snippets

This appendix provides essential excerpts from the implementations of SimCLR, MoCo, and BYOL. The complete codebase is available at: https://github.com/soumili202/Contrastive_Learning

A.1 BYOL Loss Function

```
1 import torch.nn.functional as F
2
3 def byol_loss(pred, target):
4     pred = F.normalize(pred, dim=-1)
5     target = F.normalize(target, dim=-1)
6     return 2 - 2 * (pred * target).sum(dim=-1).mean()
```

Code 1. BYOL Loss Function

A.2 SimCLR NT-Xent Loss Function

```
1 def nt_xent_loss(z1, z2, temperature=0.5):
2     z = F.normalize(torch.cat([z1, z2], dim=0), dim=1)
3     similarity = torch.mm(z, z.T)
4     n = z1.size(0)
5     mask = torch.eye(2*n, device=z.device).bool()
6     positives = torch.cat([torch.diag(similarity, n),
7                             ↪ torch.diag(similarity, -n)])
8     negatives = similarity[~mask].view(2*n, -1)
9     logits = torch.cat([positives.unsqueeze(1),
10                        ↪ negatives], dim=1)
11     labels = torch.zeros(2*n, dtype=torch.long, device=z.device)
12     return F.cross_entropy(logits / temperature, labels)
```

Code 2. SimCLR NT-Xent Loss Function

A.3 MoCo Loss and Queue Update

```
1 def moco_loss(q, k, queue, temperature=0.07):
2     l_pos = torch.einsum('nc,nc->n', [q, k]).unsqueeze(-1)
3     l_neg = torch.einsum('nc,ck->nk', [q, queue.clone()])
4     logits = torch.cat([l_pos, l_neg], dim=1)
5     labels = torch.zeros(q.size(0), dtype=torch.long).to(q.device)
6     logits /= temperature
7     return F.cross_entropy(logits, labels)
8
9 @torch.no_grad()
10 def dequeue_and_enqueue(self, keys):
11     batch_size = keys.shape[0]
12     ptr = int(self.queue_ptr)
13     self.queue[:, ptr:ptr+batch_size] = keys.T
14     ptr = (ptr + batch_size) % self.queue.size(1)
15     self.queue_ptr[0] = ptr
```

Code 3. MoCo Loss and Queue Update

A.4 SimCLR Model Architecture

```
1 class SimCLRModel(nn.Module):
2     def __init__(self, base_model, projection_dim):
3         super().__init__()
4         self.encoder = models.__dict__[base_model](
5             ↪ weights=None)
6         self.encoder.fc = nn.Identity()
7         self.projection_head = nn.Sequential(
8             nn.Linear(512, 256),
```

```
8         nn.ReLU(),
9         nn.Linear(256, projection_dim)
10    )
11
12    def forward(self, x):
13        h = self.encoder(x)
14        z = self.projection_head(h)
15        return z
```

Code 4. SimCLR Model

A.5 MoCo Model Architecture

```
1 class MoCoModel(nn.Module):
2     def __init__(self, base_encoder='resnet18',
3         ↪ projection_dim=64, queue_size=65536, momentum=0.999):
4         super().__init__()
5         self.encoder_q = models.__dict__[base_encoder](
6             ↪ pretrained=False)
7         self.encoder_q.fc = nn.Identity()
8         self.encoder_k = models.__dict__[base_encoder](
9             ↪ pretrained=False)
10        self.encoder_k.fc = nn.Identity()
11
12        self.projector_q = nn.Sequential(
13            nn.Linear(512, 256), nn.ReLU(), nn.Linear(256,
14            ↪ projection_dim)
15        )
16        self.projector_k = nn.Sequential(
17            nn.Linear(512, 256), nn.ReLU(), nn.Linear(256,
18            ↪ projection_dim)
19        )
20
21        self.register_buffer("queue", torch.randn(
22            ↪ projection_dim, queue_size))
23        self.queue = nn.functional.normalize(self.queue,
24            ↪ dim=0)
25        self.register_buffer("queue_ptr", torch.zeros(1,
26            ↪ dtype=torch.long))
27        self.momentum = momentum
28
29        self._momentum_update_key_encoder()
30
31    def _momentum_update_key_encoder(self):
32        for param_q, param_k in zip(self.encoder_q.
33            ↪ parameters(), self.encoder_k.parameters()):
34            param_k.data.copy_(param_q.data)
35            param_k.requires_grad = False
```

Code 5. MoCo Model

A.6 BYOL Model Architecture

```
1 class BYOLModel(nn.Module):
2     def __init__(self, base_encoder='resnet18',
3         ↪ projection_dim=64, momentum=0.996):
4         super().__init__()
5         self.online_encoder = models.__dict__[
6             ↪ base_encoder](pretrained=False)
7         self.online_encoder.fc = nn.Identity()
8
9         self.target_encoder = models.__dict__[
10            ↪ base_encoder](pretrained=False)
11        self.target_encoder.fc = nn.Identity()
12
13        self.online_projector = nn.Sequential(
14            nn.Linear(512, 256), nn.ReLU(), nn.Linear(256,
15            ↪ projection_dim)
16        )
17        self.target_projector = nn.Sequential(
18            nn.Linear(512, 256), nn.ReLU(), nn.Linear(256,
19            ↪ projection_dim)
20        )
21        self.online_predictor = nn.Sequential(
22            nn.Linear(projection_dim, 256), nn.ReLU(),
23            ↪ nn.Linear(256, projection_dim)
```

```

18         )
19
20         self._momentum_update_target_encoder()
21         self.momentum = momentum
22
23     def _momentum_update_target_encoder(self):
24         for param_online, param_target in zip(self.
25             ↪ online_encoder.parameters(), self.target_encoder.
26             ↪ parameters()):
27             param_target.data.copy_(param_online.data)
28             param_target.requires_grad = False

```

Code 6. BYOL Model

A.7 t-SNE Visualization Function

```

1 def plot_tsne(embeddings, labels, save_path):
2     tsne = TSNE(n_components=2, random_state=42)
3     reduced = tsne.fit_transform(embeddings)
4     plt.figure(figsize=(20, 16))
5     scatter = plt.scatter(reduced[:, 0], reduced[:, 1],
6         ↪ c=labels, cmap='tab10', s=25)
7     plt.legend(*scatter.legend_elements(), title="
8         ↪ Classes", fontsize=25)
9     plt.title("t-SNE of Learned Embeddings", fontsize=40
10        ↪ )
11     plt.savefig(save_path)

```

Code 7. t-SNE Evaluation Visualization

A.8 Linear Evaluation Protocol

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score
3
4 def linear_evaluation(model, train_loader, test_loader,
5     ↪ device):
6     model.eval()
7     model.to(device)
8
9     def extract_embeddings(loader):
10         embeddings, labels = [], []
11         with torch.no_grad():
12             for x, y in loader:
13                 x = x.to(device)
14                 z = model.encoder(x)
15                 embeddings.append(z.cpu())
16                 labels.append(y)
17         return torch.cat(embeddings).numpy(), torch.cat(
18             ↪ labels).numpy()
19
20     X_train, y_train = extract_embeddings(train_loader)
21     X_test, y_test = extract_embeddings(test_loader)
22
23     clf = LogisticRegression(max_iter=1000)
24     clf.fit(X_train, y_train)
25     y_pred = clf.predict(X_test)
26     acc = accuracy_score(y_test, y_pred)
27     print(f"Linear evaluation accuracy: {acc * 100:.2f}%"
28         ↪ )

```

Code 8. Linear Evaluation Using Logistic Regression